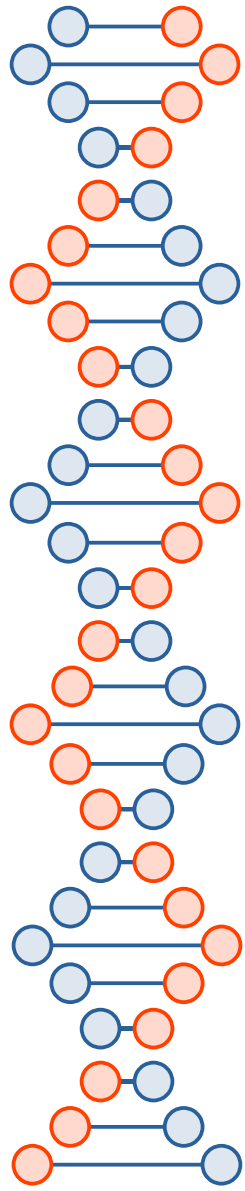


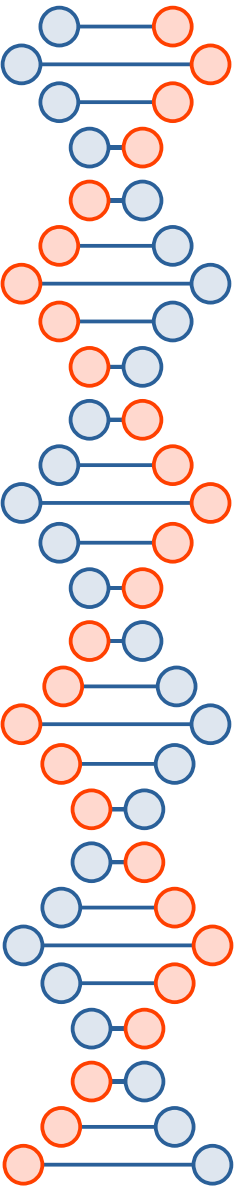
Funciones, Subrutinas y OOP en Perl

By PhD. Josafá Pontes



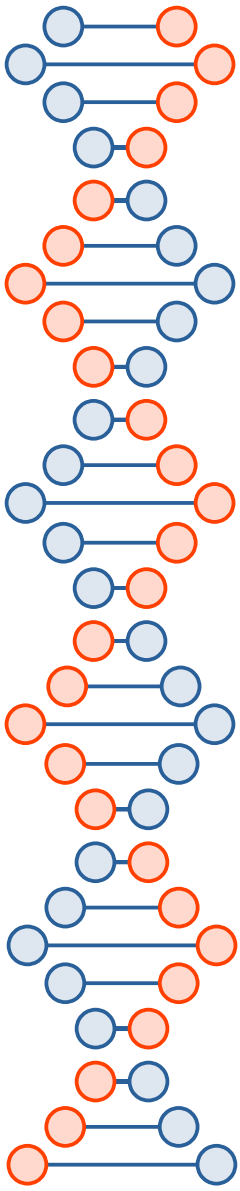
Funciones, Subrutinas y Programación Orientada a Objetos en Perl

- Curso: Introducción a Perl
- Tema: Modularidad y abstracción
- Objetivo: Escribir código reutilizable y estructurado



Funciones vs Subrutinas en Perl

- En Perl NO hay diferencia entre funciones y subrutinas
- Todo se define con sub
- Se pueden:
- Definir
- Invocar
- Pasar parámetros
- Retornar valores

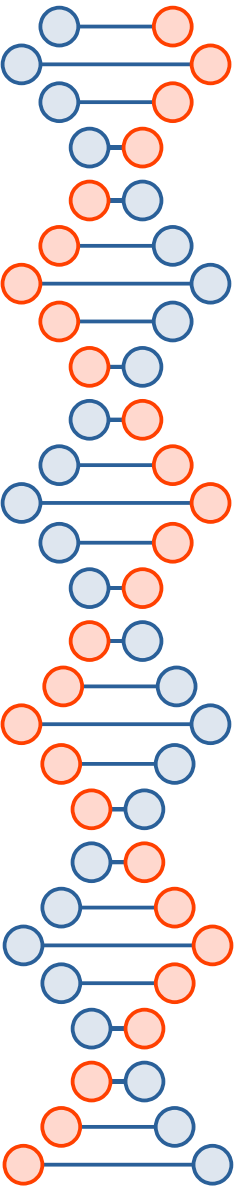


Definición básica

```
sub misubrutina {  
    print "Soy una subrutina\n";  
}
```

Característica de una subrutina:

- No requiere tipo de retorno
- Sintaxis simple

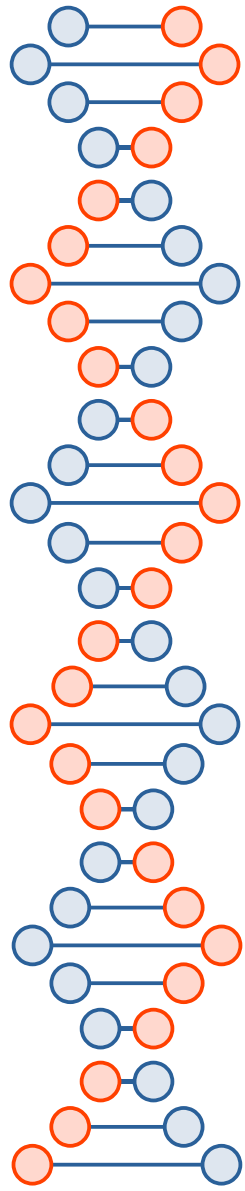


Parámetros (@_)

```
sub saludar {  
  my ($var) = @_; # my $var = shift; # cuando hay 1 solo parámetro  
  printf "Hola %s\n", $var;  
}  
saludar("Cesar"); # Hola Cesar
```

Conceptos:

- @_ contiene los parámetros
 - ▯ my (\$var) = @_;
 - ▯ \$var = shift; # \$var = shift @_;

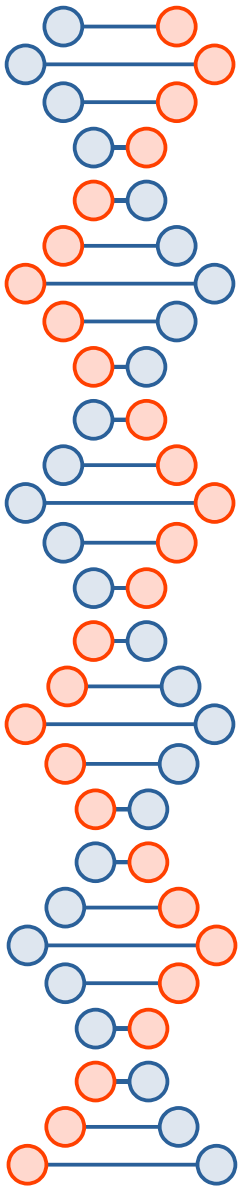


Forma alternativa de paso de parámetros (@_)

```
sub saludar {  
    printf "Hola %s\n", $_[0];  
}  
saludar("Cesar"); # Hola Cesar
```

Conceptos:

- Acceso posicional (\$_[0], \$_[1], ...)



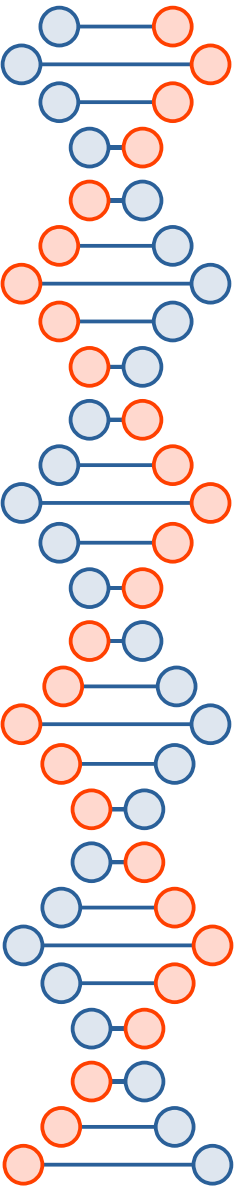
Buenas prácticas

```
sub suma {  
  my ($x, $y) = @_;  
  return $x + $y;  
}  
print suma(3, 4); # 7
```

Usar my (variables locales)

Dar nombres significativos

Evitar usar @_ directamente (\$_[0], ...) al sacar los valores de la lista @_

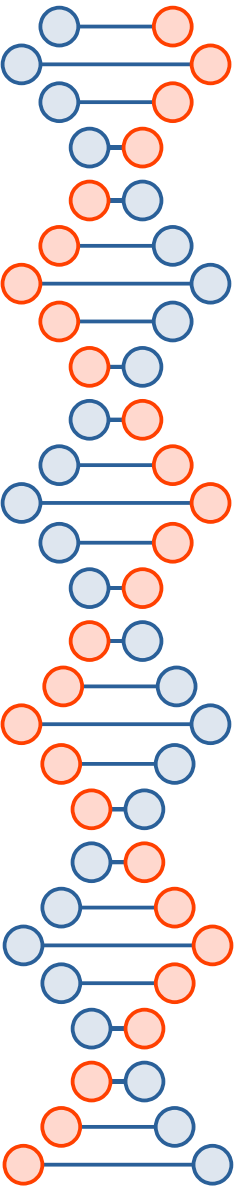


Retorno de valores en funciones

```
sub maximo {  
    my ($x, $y) = @_;  
    return $x > $y ? $x : $y; # Operador ternario  
}  
  
print maximo(3, 4); # 4
```

Nota importante:

- Si no usas return, Perl devuelve la última expresión evaluada



Parámetros tipo hash

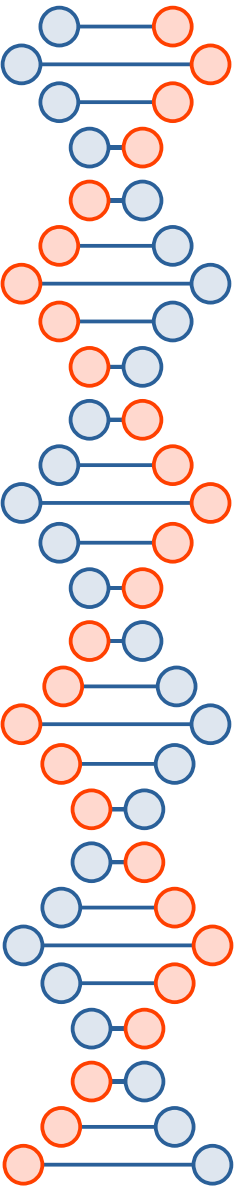
```
sub persona {  
    my %args = @_;  
    printf "Nombre: %s\n", $args{nombre};  
    printf "Edad: %s\n", $args{edad};  
}  
persona(nombre => "Maria", edad => 20);
```

Ventajas:

- Parámetros nombrados
- Más legible

Valores por defecto mediante hash

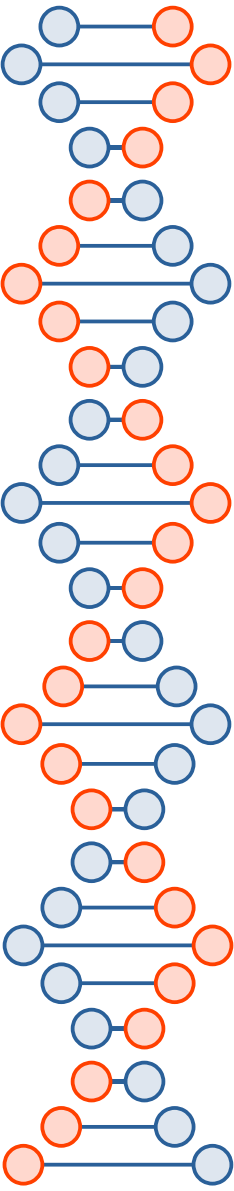
```
sub persona {  
    my %args = (enfermo => 0, @_);  
    printf "Nombre: %s\n", $args{nombre};  
    printf "Edad: \n", $args{edad};  
    printf "Enfermo: %s\n", $args{enfermo};  
}  
  
persona(nombre => "Maria", edad => 20);  
persona(nombre => "Maria", edad => 20, enfermo => 1);
```



Parámetros obligatorios vs opcionales

```
sub persona {  
    my ($nombre, $edad, %args) = (splice(@_, 0, 2), enfermo => 0, @_);  
    printf "Nombre: %s\n", $nombre;  
    printf "Edad: %s\n", $edad;  
    printf "Enfermo: %s\n", $args{enfermo};  
}  
persona("Maria", 20);  
persona("Maria", 20, enfermo => 1);
```

- Primero los obligatorios como escalares, luego los opcionales **nombrados** como hash.

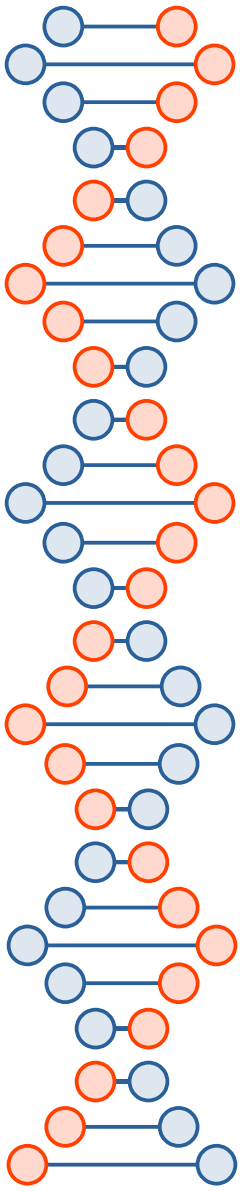


Validación de parámetros

```
if (!defined $nombre) {  
    print STDERR "Falta nombre\n";  
    return;  
}
```

Manejo de errores básico

Validación de los parámetros mediante código



Paso de parámetros por referencia - idea general

En Perl:

- Los parámetros en @_ son alias de los originales
- También podemos pasar referencias explícitas
- Permite modificar datos fuera de la función

Tipos de referencia:

- Escalar → \ \$var
- Arreglo → \@arr
- Hash → \%hash

Referencia a escalar

```
sub incrementar {  
    my ($ref_scalar) = @_;  
    $$ref_scalar += 1; # Siempre desreferenciar una referencia dentro de la función  
}  
my $x = 10;  
incrementar(\$x);  
print $x; # 11
```

Conceptos claves:

- `\$x` → referencia
- `$$ref` → desreferenciar en el interior de la función

Explicación visual (escalares)

$x = 10$



$\backslash x$ —► referencia



valor (10)

Dentro de la función:

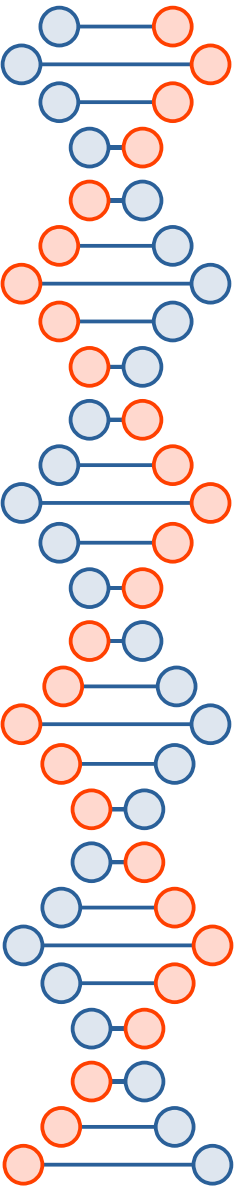
- $\$ref$ apunta a x y tengo que desreferenciar
- $\$ref$ modifica el original

Paso de arreglo indexado por referencia

```
sub duplicar {  
  my ($arr_ref) = @_;  
  foreach my $i (0 .. $#{$arr_ref}) { # El símbolo $# da el índice del último elemento de la referencia  
    $arr_ref->[$i] *= 2; # Desreferenciación del elemento del array mediante la flechita ->  
  }  
}  
  
my @nums = (1, 2, 3);  
duplicar(\@nums);  
print "@nums"; # 2 4 6
```

Claves:

- \@nums → referencia antes de invocar la función
- \$arr_ref->[\$i] → acceso mediante desreferenciación



Referencia a arreglos asociativos (hash)

```
sub actualizar_edad {  
    my ($hash_ref) = @_;  
    $hash_ref->{edad} += 1; # Desreferenciación mediante la flecha ->  
}
```

```
my %persona = (nombre => "Maria", edad => 20);  
actualizar_edad(\%persona);  
print $persona{edad}; # 21
```

Claves:

- \%persona
- \$hash_ref → {clave} # Desreferenciar siempre en el interior de la función

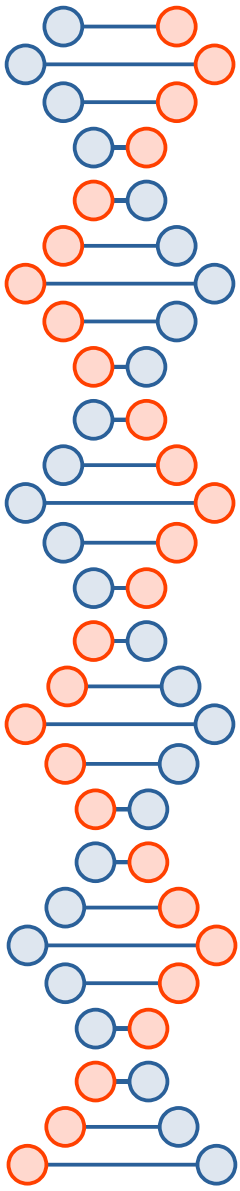
Múltiples referencias

```
sub procesar {  
  my ($x_ref, $arr_ref, $hash_ref) = @_;  
  $$x_ref += 10;  
  push @$arr_ref, 99;  
  $hash_ref->{activo} = 1;  
}
```

```
my $x = 5;  
my @a = (1, 2);  
my %h = ();  
procesar(\$x, \@a, \%h);
```

Resultado:

- \$x = 15
- @a = (1, 2, 99)
- %h = (activo => 1)



Errores comunes al pasar referencias

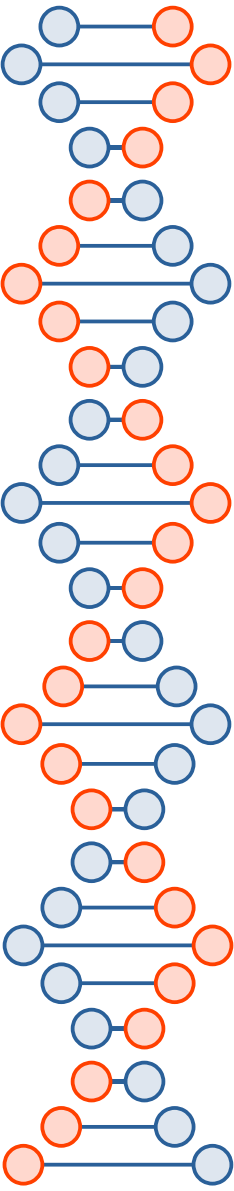
Olvidar referenciar las variables al enviarlas como parámetros:

`incrementar($x);` # ERROR conceptual

(Fatal) No desreferenciar una referencia en el interior de la función:

`$ref += 1;` # incorrecto

Correcto: Siempre desreferenciar dentro de la función
`$$ref += 1;`



Cuándo usar las referencias

Cuando:

- Quieres modificar variables externas
- Trabajas con estructuras de datos grandes
- Evitas copias innecesarias

Muy usado en:

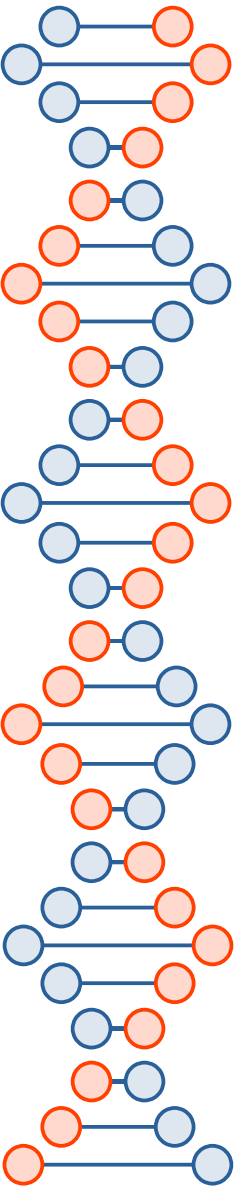
- OOP (objetos son referencias)
- Estructuras complejas

Comparación rápida

Tipo	Referencia	Desreferenciación
Escalar	<code>\\$x</code>	<code>\$\$ref</code>
Arreglo	<code>\@arr</code>	<code>\$ref->[i] # elem</code> <code>@\$ref # all list</code>
Hash	<code>\%hash</code>	<code>\$ref->{key} # elem</code> <code>%\$ref # all hash</code>

Orientación a objetos

- Clase → package
- Objeto → instanciar con new
- Métodos → subrutinas/funciones
- Atributos → hash interno

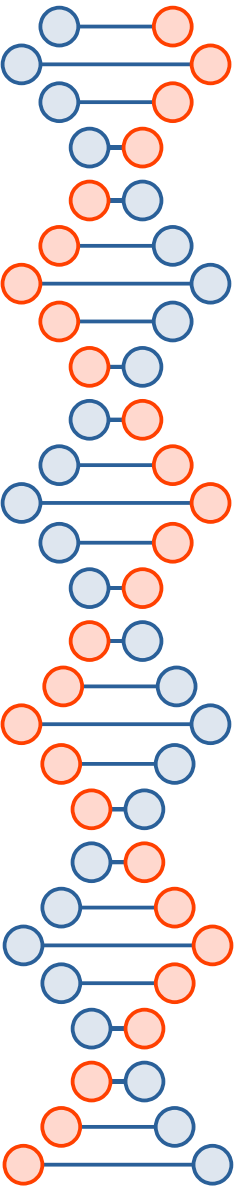


Definir una clase con su constructor

```
package Empleado{  
  
  sub new { # método constructor  
    my ($class, $nombre, $email) = @_;  
    my $self = {  
      nombre => $nombre,  
      email => $email,  
    };  
    return bless ($self, $class);  
  }  
  
  1; # return true  
}
```

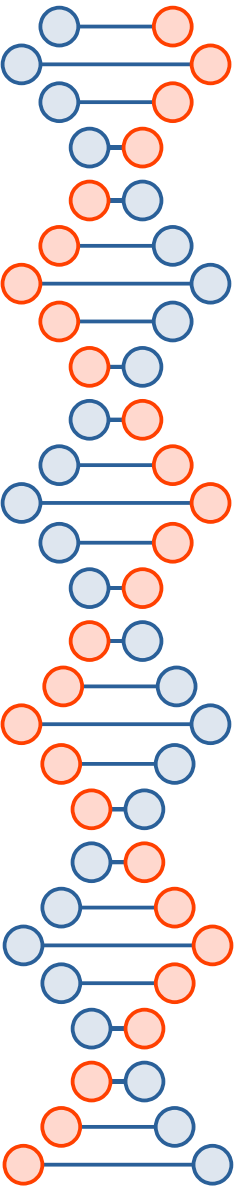
```
my $empleado = new Empleado('José', 'jose@epn');
```

la función bless convierte la referencia de hash en un objeto



Método Getter + Setter

```
sub nombre {  
    my ($self, $nombre) = @_;  
    $self->{nombre} = $nombre if defined $nombre;  
    return $self->{nombre};  
}  
  
print $empleado->{nombre}; # José
```

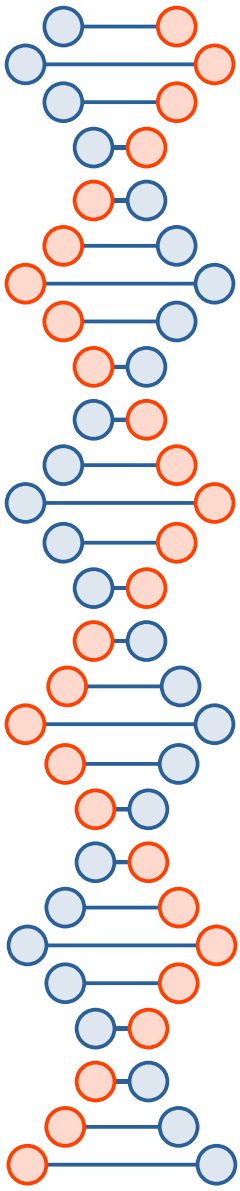
Instanciación y uso de los objetos

```
my $emp = new Empleado('Maria', 'maria@mail.com');
```

```
print $emp->nombre();
```

```
print $emp->email();
```

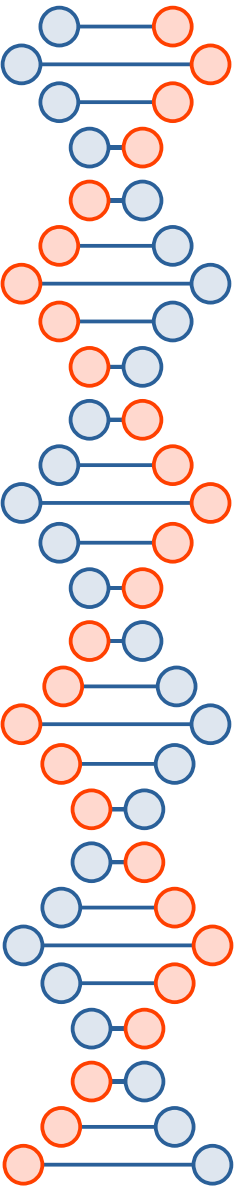
- -> para acceder a métodos



Herencia

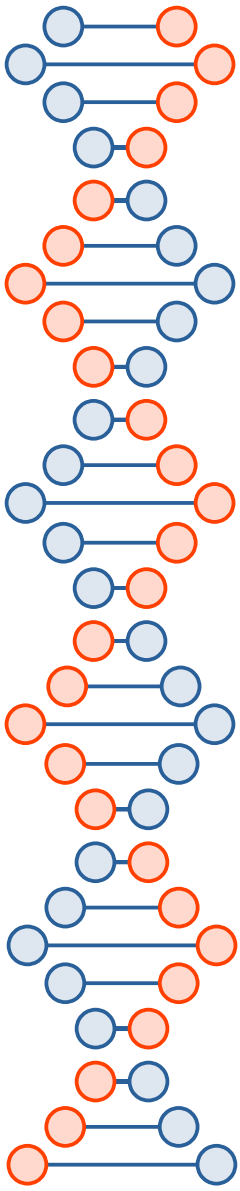
```
package Sueldo{  
    use base qw(Empleado);  
}
```

- Permite reutilizar código



Método en clase hija

```
sub sueldo {  
    my ($self) = @_; # $self = shift;  
    return $self->{dias_trabajados} * $self->{sueldo_diario};  
}
```



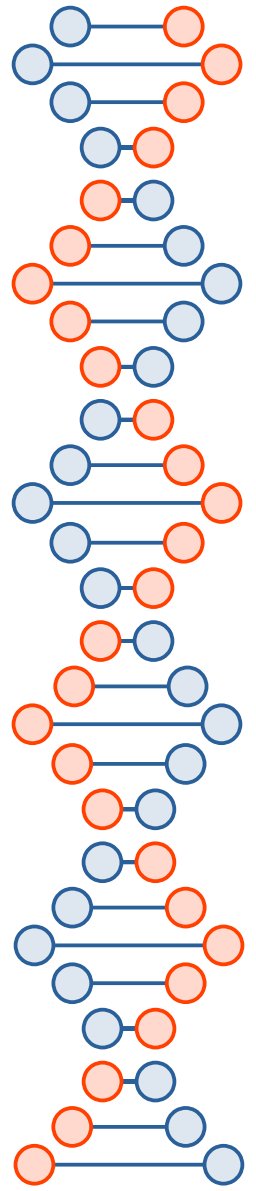
Uso de SUPER

```
$self->SUPER::metodo();
```

Accede a métodos de la clase padre

TABLA RESUMEN

Concepto	Sintaxis	Uso
Subrutina/ funciones	sub nombre { }	Definir funciones
Paso de Parámetros	@_, shift, \$_[0]	Parámetros de entrada
Retorno	return	Salida
Hash args	%args = @_	Parámetros nombrados
Clase	package	Definir una clase
Constructor	sub new + método bless	Crear constructor
Invocar método	\$obj->metodo()	Acceso al método
Herencia	use base	Reutilización de código

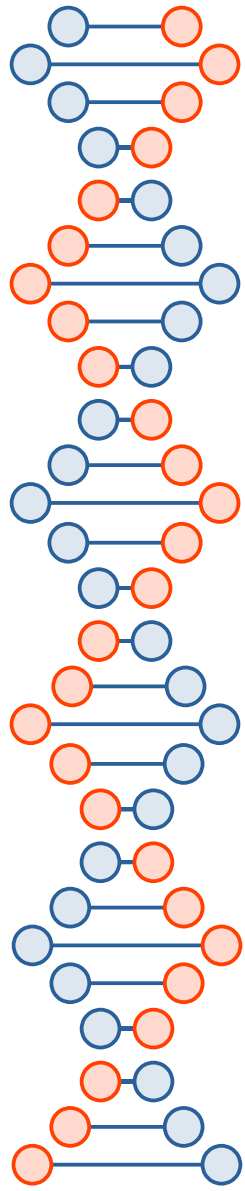


Ejercicio – Función

```
sub promedio {  
  # recibe lista de números  
}
```

Debe:

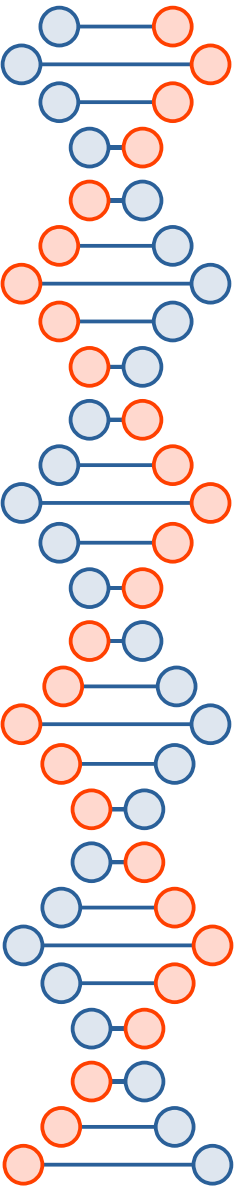
- Definir un array (@)
- Validar que todos sean números: `if ($val !~ m/^\d+(?:\.\d+)?$/)`
- Retornar el promedio
`use List::Util qw(sum);`



Paso de parámetros obligatorios y opcionales

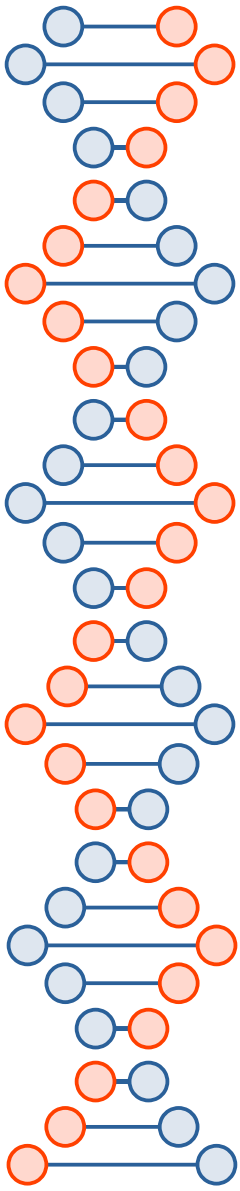
- Crear una función que reciba los parámetros obligatorios como escalares, los opcionales a través de un hash y imprimir todos los parámetros recibidos.
- Validar que los obligatorios estén definidos.

```
sub crear_estudiante {  
    # apellidos, email, y ciudad opcional  
}
```



Orientación a Objetos

- Crear clase:
package Estudiante { ... }
Debe tener:
 - atributos: nombre, nota
 - método: aprueba() → retorna true si $\text{nota} \geq 7$



Herencia

- Crear:
`package EstudianteBecado { ... }`
- Hereda de Estudiante
- Método extra: `monto_beca()`